M4 Design Justification Report — Mixed-Precision Transformer FFN Accelerator Chiplet

Course: ECE 410/510 — Hardware for Artificial Intelligence and Machine Learning, Spring 2026 **Project:** Output-stationary systolic FFN forward/backward accelerator on SAED32, with a Sky130 OpenLane PnR sweep. **Headline numbers:** 8/8 functional testbenches PASS; 2 ns / 500 MHz Genus close on SAED32 RVT (0 violators, WNS +0.1 ps); 9.07 mm^2 cell area; 2.347 W typ; 4.096 TFLOP/s peak compute.

---

1. Problem and motivation

The chip's central architectural commitment is fusion: collapse the transformer's four dominant kernels — FFN forward, FFN backward, attention forward, attention backward — into single on-chip streaming pipelines so no intermediate activation round-trips through host memory. The M1 profiling run (`codefest/cf02/profiling/project_profile.txt`) on our reference NumPy transformer (vocab 65, seq 64, d_model 64, d_ff 256, 2 layers, batch 4) shows these four kernels accounting for **~81 % of total compute time** (cumulative time) — `ff_backward` 32.5 %, `ff_forward` 23.5 %, `mha_backward` 13.7 %, `mha_forward` 11.1 % — and each is gated by **the same architectural problem**: every intermediate activation between sub-operations (matmul → bias → GELU → matmul, or matmul → softmax → matmul) gets written to DRAM and read back, because the CPU can't stage it in cache for the next sub-op. On a single-channel DDR4-3200 i5-10500H the kernels collectively achieve only 3.4 GFLOP/s sustained against a ~430 GFLOP/s peak — under 1 % utilization — and the M1 roofline shows why: at FP64 the kernels sit at AI = 5.43 FLOPs/byte, well to the left of the CPU's ridge, so each intermediate spill costs real wall-clock and real energy.

The M4 chiplet collapses each of those round trips. The four supported macro commands (`MODE_FFN_FWD`, `MODE_FFN_BWD`, `MODE_ATTN_FWD`, `MODE_ATTN_BWD`, defined in `accel_pkg.sv`) each correspond to a fused on-chip pipeline: FFN_FWD fuses $Y = \mathrm{GELU}(X \cdot W_1 + b_1) \cdot W_2 + b_2$ through `systolic → gelu_unit_lut → systolic`; FFN_BWD fuses $dh_1 = dy \cdot \mathrm{GELU}'(h_{pre})$ through `fused_postproc_unit`; ATTN_FWD/BWD fuse $\mathrm{softmax}(QK^T/\sqrt{d_k}) \cdot V$ through `systolic → causal_mask_unit → softmax_unit_lut → systolic`. In every case the intermediates never leave on-chip storage. The **payoff of fusion** is bandwidth: each round-trip saved is a chip-boundary transfer that costs interface bandwidth and per-byte energy. The host link only carries inputs and the final output per macro — three to four times fewer bytes than a step-by-step host-orchestrated version would move.

Custom silicon is justified by (a) **fusion eliminates the off-chip intermediates** that bottleneck the CPU baseline; (b) the architectural peak is 4.096 TFLOP/s vs the CPU's 139 GFLOP/s attainable at this AI — **29.5× peak / CPU-attainable**; (c) at peak compute, energy is **0.57 pJ/FLOP** vs ~260 pJ/FLOP for the CPU — two orders of magnitude better; (d) the array dimension and accumulator width are parameterized so the same RTL scales without firmware changes. What this report shows (and §8 quantifies) is that the **measured** delivered numbers fall well short of those architectural numbers, despite the chip closing timing cleanly and verifying correctly. **§9 explains why the gap exists and what would close it.**

2. Roofline analysis

The roofline plot in `bench/roofline_final.png` shows three ceilings and two measured points.

- **CPU (M1 platform)**: peak ~430 GFLOP/s, DRAM line 25.6 GB/s. FFN_BWD at AI = 5.43 sits below the ridge — memory-bound on this platform.
- **M4 external-channel roof**: peak 4.096 TFLOP/s (4,096 PEs × 500 MHz × 2 FLOPs/MAC), bounded by the chiplet-boundary bandwidth of 2 GB/s (one 32-bit **Q16.16 word per cycle** on the UCIe-style read- and write-data channels). On this ceiling the ridge is at AI = 2,048 — to the right of any practical FFN/attention kernel. **Everything that crosses the chip boundary is memory-bound on this ceiling.**
- **M4 on-chip SRAM roof**: same peak, with 128 GB/s tile-buffer bandwidth (64 Q16.16 elements per cycle × 4 B × 500 MHz feeding the systolic edges). Ridge at AI = 32. Once tiles are resident in `tile_buffer`, the systolic is fed at full bandwidth and the architecture is compute-bound for any reuse-positive kernel.

All bytes are counted at the **Q16.16 wire/scratchpad format (4 B/element)** — the format that moves through the chip's memory hierarchy until each PE's Q4.4 input quantizer at the MAC. The two measured points are:

- **M1 baseline**: AI = 5.43 (FP64 8 B/element), attained 3.4 GFLOP/s — within 4× of the DRAM ceiling, where a NumPy implementation lives.
- **M4 measured (single 64×64×64 FFN_BWD tile)**: AI = **8.12** (Q16.16 4 B/element, derived as 532,480 FLOPs / 65,536 B over the chiplet boundary), attained **8.07 GFLOP/s**.

The M4 point sits nearly *on* the external-channel roofline at AI = 8.12, where the line yields `2 GB/s × 8.12 = 16.24 GFLOP/s`. The chip therefore runs at **~50 % of its interface ceiling** today. It is **not** sitting orders of magnitude below the SRAM roof for plumbing reasons; it is sitting close to the *interface* roof because the external bandwidth is what limits the kernel as plotted. The SRAM roof at AI = 8.12 still gives 1.04 TFLOP/s (128 × 8.12), so the chip has ~100× of *internal* compute headroom that the boundary cannot deliver to it. **§9 isolates which bottleneck is which and what would lift each one.**

The interface ceiling is the central architectural finding of M4. At FFN_BWD's AI = 8.12 the 2 GB/s chiplet boundary caps sustained throughput at 16.24 GFLOP/s — `4096 / 16.24 ≈ 250×` below the on-chip compute peak. The 1 TFLOP/s SRAM roof and 4 TFLOP/s compute peak are only reachable for workloads that *change* the effective AI at the chip boundary (model-resident inference, weight-stationary FFN_FWD, or quantize-on-the-wire — each detailed in §9). The bottleneck shifts platform-to-platform: on the CPU the kernel is DRAM-bound; on the accelerator it becomes interface-bound rather than compute-bound, because the architecture explicitly trades off-chip bandwidth for on-chip reuse via fusion.

Why the AI is limited to 8.12 at all. The kernel's *intrinsic* reuse is much higher than 8 FLOPs per byte — the 64×64 systolic touches each loaded operand 64 times during the K reduction, giving an internal reuse factor of 128 FLOPs per Q4.4 multiplier-input byte. The boundary AI is forced down to 8.12 by three concrete properties of the chip *as built*, in order of impact:

1. **Wire format mismatch.** `interface.sv:120` transmits operands as Q16.16 (4 B/element), but the multiplier only consumes Q4.4 (1 B/element). Three of every four bytes crossing the boundary are zero-padding the chip discards. This alone is a 4× AI penalty — fix #6 in §9.

2. **No cross-macro reuse of operands.** Every FFN_BWD macro reloads A, B, and AUX from scratch even when consecutive macros would share weights (FFN_FWD) or sequences (attention). The chip has a 512 KB scratchpad that could hold an entire FFN layer, but no operand-pinning protocol — fixes #7 and #8 in §9.
3. **Output written back per macro.** Each macro's 16 KB output tile crosses the boundary even though the next pipeline stage in a real model would consume it on-chip if the next macro were fused with this one — partially addressed by fix #1 (multi-tile macros) in §9.

§9 quantifies each fix's AI delta cumulatively: fix #6 alone raises AI 8.12 → 32.5; fix #8 raises it past 500; the combination puts the chip in the compute-bound regime where the 4 TFLOP/s peak is reachable.

3. Precision and data format

The datapath uses a **mixed-precision Q4.4 / Q16.16 scheme** modelled after NVFP4's small-multiplier / wide-accumulator philosophy. Operand registers, scratchpad cells, and inter-PE forwarded values are all **Q16.16 signed fixed-point (32-bit)**. At each PE, the multiplier inputs are quantized to **Q4.4 (8-bit signed)** via arithmetic right shift by `Q44_ALIGN_SH = FRAC_BITS − MULT_FRAC` with saturation. The 8×8 multiplier produces a **Q8.8 (16-bit)** product which is sign-extended and left-shifted into Q16.16 for the accumulator. See `mac_pe.sv` header (lines 1–30) and `accel_pkg::q_mul / q_add_sat`.

This was chosen for three reasons. **Range** — the M2 acceptability study (`m2/precision.md`) showed transformer pre-activations never exceed ±100 in our reference run; Q16.16's ±32 K range is far past that, eliminating per-block exponent bookkeeping that FP16/BF16 would need. **Multiplier area** — an 8×8 multiplier is $\sim4\times$ smaller than 16×16 and $\sim16\times$ smaller than 32×32; with 4,096 PEs that difference dominates chip area. The Genus post-synth area report (`synth/area_report.txt`) shows the systolic array at 8.51 mm^2 (out of 9.07 mm^2 total), confirming that the multiplier choice is the area driver. **Error tolerance** — the M2 precision sweep (`m2/sim/precision_results.txt`) measured per-kernel MAE: GEMM 2.5×10^{-5}, Softmax 8×10^{-5}, GELU 1×10^{-2} absolute, GELU′ 2×10^{-2} absolute over 100 random samples per kernel. The GELU error is the Padé tanh approximation's intrinsic error, not Q-format quantization noise; FP32 with the same Padé shows the same number. End-to-end `tb_compute_core` passes against an FP32 golden model with a 10 % absolute tolerance, and the LUT replacements in M4 (256-entry direct LUT + linear interp for GELU and GELU′) drive the activation error below 1 %.

Quantization acceptability is therefore established two ways: per-kernel (M2 precision study) and end-to-end (`tb_compute_core: PASS` in `sim/final_run.log` and `tb_ff_backward_e2e: PASS`).

4. Dataflow and architecture

The accelerator is an **output-stationary systolic** design. The 64×64 systolic array (`systolic_array_64x64.sv`) holds one partial-product accumulator per PE and walks the inner-product dimension K through the array using west-to-east and north-to-south register forwarding. Operand A streams in from the west, operand B from the north; at each cycle every PE updates its accumulator with `acc += a × b`. At the end of K cycles the accumulators hold one 64×64 GEMM output tile. This is the classical TPU dataflow; output-stationary fits the FFN kernel because each output element is touched many times during reduction and only once for writeback, so keeping it pinned to the PE eliminates the read-

modify-write loop that would otherwise dominate.

Around the array, the chip contains:

- **`compute_core.sv`** (M2 unchanged): 16 data-parallel lanes (`N_LANES=16`), each with its own scratchpad and accelerator instance. The top-of-system dispatcher (`tile_dispatcher.sv`) takes one `macro_cmd_t` and round-robins tiles across lanes.
- **Per-lane `accel_engine.sv`**: ties one systolic array to a `stream_pipeline.sv` (matmul → optional GELU/GELU′ post-processing → softmax for attention) and a `fused_postproc_unit.sv` (the FFN-backward `dh = dy · GELU′(h_pre)` multiplier).
- **Memory hierarchy**: per-lane scratchpad `sram_bank.sv` (32 KB) → tile loader/double-buffer (`tile_loader.sv`, `double_buffer_ctrl.sv`, `tile_buffer.sv`) → on-PE accumulator. The tile loader walks the scratchpad in row-major order and presents one Q4.4 element per cycle to the systolic edges, so the array sees 64 + 64 new operands every cycle when full.
- **Activations**: a 256-entry direct LUT + linear interpolation replaces the M2/M3 Padé tanh chain. `gelu_unit_lut.sv` and `gelu_grad_unit_lut.sv` drop both area and the GELU/GELU′ MAE below 1 %. `softmax_unit_lut.sv` uses an exp LUT plus a sequential 1/sum reciprocal (`divider_or_reciprocal_seq.sv`).
- **Control plane**: `accel_controller.sv` is the macro-level state machine. It accepts one `macro_cmd_t` over the UCIe command bus, sequences the loads (A, B, AUX for FFN_BWD) and the store (output tile), and asserts `irq` when the macro completes.
- **External face**: `interface.sv` (M2 unchanged) terminates the UCIe-style host link (cmd, write, read channels + irq) and presents them as `core_*` valid/ready handshakes that mate one-to-one with `compute_core`.

Three M5/M6 refactors moved the design forward without changing the architecture: the divider was retimed into the sequential reciprocal path (`divider_or_reciprocal_seq.sv`), the MAC was deepened from 1-cycle (`mac_pe.sv`) to 4-cycle pipelined (`mac_pe_piped4.sv`) to break the post-place critical path, and the softmax row capture window was decoupled from the row arrival cadence using a backpressure handshake (`stream_pipeline.sv` `ready` signal) to fix the M3 integration failure.

5. Hardware interface

The chiplet's external face is a **UCIe-style five-channel link** terminated by `interface.sv`. The five channels are a 128-bit packed `macro_cmd_t` command stream (one packet per macro), a 51-bit write-data stream (19-bit address + 32-bit Q16.16 data, one element per cycle), a 32-bit read-data stream (one element per cycle), a one-bit interrupt (`irq`, asserted on macro completion), and a busy bit. All five run on the chiplet clock (500 MHz at the SAED32 close); a real UCIe PHY would add a CDC outside this module.

Effective bandwidth at the boundary. At 500 MHz the write channel sustains 32 bits/cycle = 2 GB/s into the scratchpad; the read channel sustains 2 GB/s back to the host. The on-wire format is Q16.16 (4 B/element), not the multiplier's on-chip Q4.4 (1 B/element), so the link carries 4× more bytes per operand than the MAC actually consumes. The command channel is bursty (one 128-bit packet per macro) and contributes nothing to steady-state bandwidth.

Per-macro transfer for a 64×64 FFN_BWD tile (Q16.16 on the wire): three input tiles A, B, AUX at 16,384 B each plus one output tile back at 16,384 B = **65,536 bytes / 33 µs total bus time** at 2 GB/s. The 32,989-cycle / 66-µs measurement reported in §8 is for cmd-issue →

IRQ only — it does **not** include the host's pre-load phase or post-read phase, both of which happen over the same UCIe-side ports and are gated by the same 2 GB/s. From the host's wall-clock perspective the full macro is closer to **99 µs**, of which 33 µs is interface and 0.4 µs is compute.

Interface-bound ceiling at FFN_BWD's AI = 8.12. Counting Q16.16 bytes on the wire (the format `interface.sv` actually transmits — `ucie_wr_data[31:0]` is a 32-bit Q16.16 word per cycle), the chiplet boundary's roofline contribution is `2 GB/s × 8.12 FLOPs/byte = **16.24 GFLOP/s**` sustained — the maximum throughput **any** workload can reach for this kernel over this link, regardless of how perfect the on-chip pipeline is. The same calculation puts FFN_FWD (AI ≈ 10.8), ATTN_FWD (AI ≈ 5), and ATTN_BWD (AI ≈ 6) within a **10–22 GFLOP/s** band. The chip's 4.096 TFLOP/s peak therefore cannot be approached by any of the four supported modes over this interface. The boundary was sized as a single UCIe sub-link (32-bit Q16.16 at 500 MHz) for the chiplet demonstrator scope; the consequence — that the compute is over-provisioned by **∼250×** relative to the link at FFN_BWD's AI — is unpacked in §9. The measured 8.07 GFLOP/s is already **∼50 % of the 16.24 GFLOP/s interface ceiling** — much closer to interface-saturated than compute-saturated.

6. Verification

The verification flow is a tiered testbench suite on QuestaSim 2021.3_1, driven by `m3/sim/run_verification.sh`, with eight self-checking testbenches:

Tier	Testbench	Scope	Result
Leaf unit	`tb_fused_postproc_unit`	`dh = dy · GELU′(h_pre)` post-processor	PASS — 3 s
Leaf unit	`tb_gelu_unit_lut`	256-entry LUT GELU vs FP32 golden	PASS — 2 s
Leaf unit	`tb_gelu_grad_unit_lut`	256-entry LUT GELU′ vs FP32 golden	PASS — 2 s
Leaf unit	`tb_softmax_unit_lut`	Sequential softmax with exp LUT + reciprocal	PASS — 2 s
Subsystem	`tb_stream_pipeline_tile`	matmul → GELU → softmax stream pipeline	PASS — 8 s
Chip	`tb_compute_core`	Four scenarios: FFN fwd, FFN bwd, back-to-back fwd↔bwd, FFN bwd at h = 2.0	PASS — 11 s
Chip	`tb_top`	UCIe-side host driving one 64 × 64 FFN forward macro	PASS — 9 s
Chip	`tb_ff_backward_e2e`	UCIe-side host driving four FFN backward macros (h = 1, 2, −1, 0)	PASS — 12 s

8/8 PASS confirmed in `sim/final_run.log`. The chip-level testbenches (`tb_compute_core`, `tb_top`, `tb_ff_backward_e2e`) drive the design only through its UCIe-side ports — host loads operand tiles via the write channel, issues the macro packet via the command channel, polls for `irq`, then samples output cells via the read channel. There is no white-box access into `compute_core` or below, so a PASS at these tiers is also a contract test that nothing inside the chip leaks state to the host. Each testbench computes a per-element FP32 reference inline and asserts agreement within a 10 % absolute tolerance,

except for `tb_ff_backward_e2e` which quantizes the reference's input through the same Q4.4 path the LUT sees, so its comparison is to 4 decimal places.

Four RTL bugs were found and fixed during the M3→M4 verification campaign and are captured in git history (commits `199c40d`, `b2354fe`, `f7e8605`, `dbdab93`): (1) `accel_controller.sv` had a 12-bit `load_cnt` / tile size that truncated `tile_out_size = 64×64 = 4096` to zero in 12 bits — fixed by widening to 13 bits; (2) `mac_pe_piped4.sv` had an accumulator-forwarding hazard where the bypass mux didn't observe the cleared-accumulator phase; (3) `fused_postproc_unit.sv` had a mismatched data delay between the GELU and GELU′ paths (4 cycles for LUT vs 6 cycles for Padé); (4) `softmax_unit_lut.sv` + `stream_pipeline.sv` were missing a backpressure handshake, causing the softmax capture FIFO to overrun for large row counts.

7. Synthesis results

Final headline: Genus 21.12 synthesis on SAED32 RVT (typ corner, balanced-tree wireload, `tt0p85v25c`), 2 ns clock period — **closes timing with 0 violators**, WNS +0.1 ps, TNS 0 ps (see `synth/qor_report.txt`).

Metric	Value	Source
Process / library	SAED32 RVT, `tt0p85v25c`	`synth/area_report.txt` header
Clock period	2.000 ns (500 MHz)	`synth/qor_report.txt`
WNS / TNS / violators	+0.1 ps / 0 / 0	`synth/qor_report.txt`
Total cell area	**9,069,382 μm² ≈ 9.07 mm²**	`synth/area_report.txt`
Leaf cell count	2,372,162	`synth/qor_report.txt`
Sequential cells	596,183 (25.1 %)	`synth/qor_report.txt`
Combinational cells	1,775,979 (74.9 %)	`synth/qor_report.txt`
Power (vectorless typ)	**2.347 W** total	`synth/power_report.txt`
Power breakdown	Internal 80.4 %, leakage 11.3 %, switching 8.3 %	`synth/power_report.txt`
Group breakdown	Registers 69.4 %, logic 30.6 %	`synth/power_report.txt`
Synthesizer runtime	25,029 s (~7 h) on phobos	`synth/qor_report.txt`

Dominant area contributor: the 64×64 systolic array `u_array` is 8.51 mm² out of 9.07 mm² total — **93.8 %** of cell area. Inside the array, each `mac_pe_piped4` instance is ~2.08 mm² × 4,096 PEs = 8.50 mm². The remaining 6.2 % is split among the `stream_pipeline` post-processing chain (softmax + GELU LUT + fused postproc), the `accel_controller` FSM, and per-lane glue. The conclusion is that **area is dominated by the multiplier-array silicon**, which validates the M2 choice of Q4.4 inputs / 8×8 multiplier — a 16×16 multiplier would have grown the array ~4× and the chip would exceed 30 mm².

Dominant power contributor: registers are 69.4 % of total power (1.63 W of 2.35 W), and the systolic dominates the register count. This is consistent with the area breakdown: the array's per-PE accumulator + 4-stage MAC pipeline carries the FF count. Clock-gating would be the natural next reduction target — the macro window is ~10 % "real compute" and 90 % data movement, so register clock-gating during fill/drain phases would cut active power

proportionally.

On Sky130 via OpenLane (`synth/config.json`, `synth/openlane_run.log`, `synth/openlane_summary.md`): a parallel Sky130 OpenLane 2.3.10 flow was run on the hand-flattened Verilog under `m3/synth/v_hand/` to satisfy the M4 OpenLane-configuration deliverable. The build is the **scoped-down `top_small`** (TILE_DIM = 2, N_LANES = 1 — 2 × 2 systolic with 1 lane) so the open-source flow completes in reasonable wall-clock. The completed run produces a **clean GDS** (Magic/KLayout DRC 0 errors, LVS 0 errors, XOR 0 differences) and **closes timing at nom_tt_025C_1v80 with WNS +0.98 ns at a 10 ns target** (100 MHz). Cell area is 0.578 mm^2 — within 21 % of the prior Attempt 8b snapshot (0.45 mm^2), confirming the M3→M4 verified RTL fixes did not regress the Sky130 PnR. The closed-timing headline numbers (9.07 mm^2, 2 ns, 4.096 TFLOP/s peak) come from the **Genus SAED32 run on the full-scale 64 × 64 / 16-lane design**, not from this Sky130 build.

8. Benchmark results

The M1 software baseline and the M4 accelerator measurement are both in `bench/benchmark.md` and the raw values in `bench/benchmark_data.csv`. To summarize:

Number	M1 SW (i5-10500H)	M4 accelerator (SAED32, 500 MHz)	Ratio
Sustained on dominant kernel	3.398 GFLOP/s (full transformer iter)	8.07 GFLOP/s (single 64 × 64 FFN_BWD tile)	**2.4**×
Architectural peak / attainable	139 GFLOP/s (CPU roofline attainable, DRAM-bound)	4,096 GFLOP/s (4,096 PEs × 500 MHz × 2)	**29.5**×
Energy per FLOP (peak)	~260 pJ/FLOP (estimate)	**0.57 pJ/FLOP**	**~450**×
Energy per FLOP (per-tile measured)	(same)	291 pJ/FLOP	~1×
Wall time (full transformer iter)	35.317 ms median	not directly comparable	—
Wall time (one 64 × 64 FFN_BWD tile)	not measured directly	65.978 µs (32,989 cyc × 2 ns)	—

The disappointing line in this table is the **measured sustained number: 8.07 GFLOP/s, only 2.4× the M1 CPU baseline** despite a silicon peak that is ~1,200× the CPU baseline (4,096 GFLOP/s peak ÷ 3.4 GFLOP/s measured). At face value the chip is delivering 0.2 % of its own peak. The two derived rows below the measured line — energy/FLOP at peak (0.57 pJ) vs energy/FLOP measured-on-tile (291 pJ), a 510× gap — capture the same problem from the energy side: the chip's energy efficiency is excellent **at the rate it is designed to run**, not at the rate it actually ran on the verified workload. The 65.978 µs macro window contains roughly 128 ns of full-array compute (the 64-cycle K reduction at the heart of the systolic) and ~65.8 µs of supporting activity around it. Beyond this single-tile measurement is the **chiplet-boundary roofline ceiling of 16.24 GFLOP/s** at this kernel's AI = 8.12 (derived in §5) — the measured 8.07 GFLOP/s is already half of that hard ceiling, so the chip is much closer to interface-saturated than it looks if you only compare it to the 4 TFLOP/s compute peak. The full chain of why the measured number is so much smaller than peak, what part of the gap is internal-plumbing-limited and what part is interface-limited, and what would close each, is the subject of §9.

9. What did not work

Several concrete things failed during M3–M4 and shaped the final design.

Data plumbing

Too much wall-clock was spent on long-hour Genus and Innovus runs (~7 h Genus + multi-hour Innovus attempts, repeatedly) trying to squeeze more out of the *single-tile compute path* — when the right investment would have been to widen the data-movement plumbing around the systolic array. The 64×64 array is the headline silicon, but on a single 64×64 FFN_BWD tile it is actively computing for only **192 of the 32,989 macro cycles (~0.6 %)**. The other 99.4 % is one-element-per-cycle data movement through the DMA, postproc, and writeback path while all 4,096 PEs sit idle. That's why the measured number is **8.07 GFLOP/s vs the 4,096 GFLOP/s peak** — the silicon is correctly architected for compute parallelism, but its plumbing was sized for single-tile correctness, not sustained throughput. Four follow-up fixes would close most of that gap; none of them require deeper logic levels, so the 2 ns close holds.

#	Fix	RTL touch	Synth impact	What it removes
1	**Multi-tile macros**	Small. `macro_cmd_t` already carries `num_m_tiles × num_n_tiles`; the controller FSM and `address_gen.sv` need to walk the tile counter correctly. Mostly verification + a few extra counters.	Negligible (~ + 0.1 % area).	Lets fill/drain (192 cyc) amortize across many tiles instead of being paid once per tile.
2	**Overlap load with compute (double-buffer)**	Medium. `double_buffer_ctrl.sv` exists, but `accel_controller` waits for load before issuing compute. Refactor into two concurrent FSMs sharing a ping-pong handshake; instantiate a 2nd set of tile buffers.	+ 5–8 % area, + 5 % power, no critical-path change.	The 8,192–12,288 cycles of tile-A/B/AUX load that today block compute.
3	**Parallel postproc (64-lane GELU′)**	Big. Parameterize `fused_postproc_unit.sv` + the GELU LUT modules with `POSTPROC_WIDTH=64`; widen the streaming pipeline to 64 elements/cycle.	**+ 1–2 mm² (~10–20 % of total area)**. Single-stage LUT + Q4.4 mul, so still fits in 2 ns.	The 4,096-cycle one-element-per-cycle postproc stream becomes ~64 cycles.
4	**Wider scratchpad load / writeback (64 elements/cycle)**	Big. `scratchpad_ctrl.sv`, `dma_engine.sv`, `address_gen.sv`, `tile_loader.sv`, `tile_writer.sv` all assume 1 elem/cycle. Need 64-bank parallel reads, 64-way address gen, 64-wide DMA.	**+ 1–2 mm² (scratchpad + addr-gen)**. Shift-and-add address gen fits in 2 ns; SRAM access unchanged.	The ~8 k cycles of scratchpad load and ~4 k cycles of writeback collapse to ~64 cycles each.

Combined impact of fixes 1–4 alone (interface-bound). Area grows 9.07 mm² → ~11.5–13 mm² (+ 25–40 %). Power grows 2.35 W → ~3.0–3.3 W. Timing still closes at 2 ns (none of these fixes deepen the critical path). Internal cycle accounting per tile collapses from ~33

k cycles to ~600 single-tile, and to ~64–80 per tile in multi-tile steady-state. **But the chip cannot exceed ~16 GFLOP/s sustained even with perfect internal plumbing — the chiplet boundary itself becomes the next ceiling, and it is already at 50 %.**

Interface as the architectural ceiling

The UCIe-style host link in `interface.sv` runs **one 32-bit Q16.16 word per cycle each direction at 500 MHz = 2 GB/s** (`interface.sv:120`). For one 64×64 FFN_BWD macro the host pushes three Q16.16 input tiles in (A, B, AUX = 49,152 B → 24.6 µs) and pulls one output tile back (16,384 B → 8.2 µs); **the host-visible per-macro cost is ~33 µs of interface time on top of 0.4 µs of compute**. The 32,989-cycle / 66-µs measurement covers cmd-issue → IRQ only, hiding the interface in pre-load and post-read phases the testbench doesn't measure.

At FFN_BWD's **AI = 8.12 FLOPs/byte** (Q16.16 wire), the roofline-on-the-chiplet-boundary gives `2 GB/s × 8.12 = **16.24 GFLOP/s**` — the hard architectural ceiling. The SRAM roof at the same AI is 1,040 GFLOP/s; the compute peak is 4,096 GFLOP/s. **The chip is over-provisioned in compute by ~250× relative to its host interface.** Fixes 1–4 lift the measured 8.07 GFLOP/s only as far as the ~16 GFLOP/s interface line (~2× over today) and no further. Reaching the SRAM ridge or the compute peak requires *changing the effective AI at the boundary*. Four more fixes, none requiring new compute silicon, do that:

#	Fix	RTL touch	What it unlocks
5	**Wider UCIe (multi-sub-link)**	Parameterize `interface.sv` with `N_UCIE_LANES`; widen the `wr_data` / `rd_data` paths and add lane-aware address routing into `dma_engine` + `scratchpad_ctrl` (most of which is already needed for fix #4). `compute_core` internal logic, the systolic array, the controller, and the cmd channel are all untouched.	16-lane UCIe → 32 GB/s. Lifts interface roof from 16 to **260 GFLOP/s** at AI = 8.12 (= 32 × 8.12). Linear in lane count.
6	**Quantize on the wire (Q4.4 vs Q16.16)**	Pack/unpack at the UCIe boundary in `interface.sv`. ~negligible silicon. No numerical impact — the on-chip multiplier already quantizes to Q4.4; we just stop *carrying* precision the link cannot use.	Free **4× AI gain**: 8.12 → 32.5 FLOPs/byte (Q4.4 wire). Interface roof: 16 → **65 GFLOP/s** on a single-lane channel, or **~1 TFLOP/s** when combined with fix #5.
7	**Weight-stationary FFN_FWD**	State bit in `accel_controller` + scratchpad tag. The B matrix in forward FFN is the *weight*, reused across every batch and sequence position; reloading it per macro is wasted bandwidth.	Cuts FFN_FWD's per-macro load bandwidth by ~2×, **doubling effective AI** to ~20 FLOPs/byte (Q16.16). Does not help FFN_BWD (needs fresh `h_pre`).

8	**Model-resident on-chip storage**	Scratchpad partition policy. Push the model in once at boot, stream only activations per inference. The current scratchpad is 32 KB × 16 lanes = 512 KB — enough for a full 64×256 FFN layer per lane.	Pushes effective AI from 8.12 to **>500 FLOPs/byte** (only activations cross the boundary). Combined with fixes 5+6, lifts sustained throughput into the **~1–4 TFLOP/s** band — the compute-peak regime.

How each fix moves AI and the interface ceiling (cumulative, from today's 8.12 / 16.24 GFLOP/s):

Fix	AI (FLOPs/B)	Interface ceiling	Mechanism
+1–4 plumbing	8.12	16.24 GFLOP/s	AI unchanged; measured climbs to ~16.
+6 Q4.4 wire	**32.5**	65 GFLOP/s	4× more FLOPs per byte over the link.
+5 16-lane UCIe	32.5	1,040 GFLOP/s	Boundary BW 2 → 32 GB/s.
+7 weight-stationary	~65 (FFN_FWD)	2,080 GFLOP/s	B reused across macros.
+8 model-resident	**>500**	interface no longer binding	Only activations cross; ~4 TFLOP/s peak reachable.

Combined. Fixes 1–6 put the chip at **~1 TFLOP/s sustained on FFN_BWD** — ~300× the M1 baseline and within striking distance of the SRAM ridge. Adding 7–8 lets FFN_FWD approach the 4 TFLOP/s compute peak for model-resident workloads.

The honest lesson for the report. The hours spent on long-wall-clock Genus + Innovus runs trying to polish the *single-tile compute path* were misallocated. The single-tile path was correct from M2; what was sized wrong was the data movement around it — first inside the chip (fixes 1–4, ~24 % more area, lifts measured to the interface roof) and then across the chip boundary (fixes 5–6, ~1 % more area in the I/O ring, lifts measured to the SRAM roof). Done in the order 1 → 2 → 3 → 4 → 5 → 6, each fix is the next bottleneck after the previous one is removed; out of order, silicon is wasted on capacity that nothing else can feed. The 4 TFLOP/s peak is reachable, but only for workloads that change the effective arithmetic intensity (fixes 7–8), not for the M1 reference workload.

Toolchain and synthesis failures

Innovus CTS broke timing. Post-place WNS = −4.96 ns; CTS crashed (`IMPCCOPT-1013`); rerun produced empty post-CTS reports. Headline is the **Genus front-end close**, not a post-PnR number.

Word count: 4,965. Sources cited: M1 `sw_baseline.md`, M2 `precision.md` + `precision_results.txt`, M3 `synthesis_notes.md`, M4 `synth/{qor,area,power,timing}_report.txt`, M4 `sim/final_run.log`, `bench/benchmark_data.csv`. Reproducibility instructions in `project/m4/README.md`.

Figures

Figure 1 -- Roofline (§2, §8).

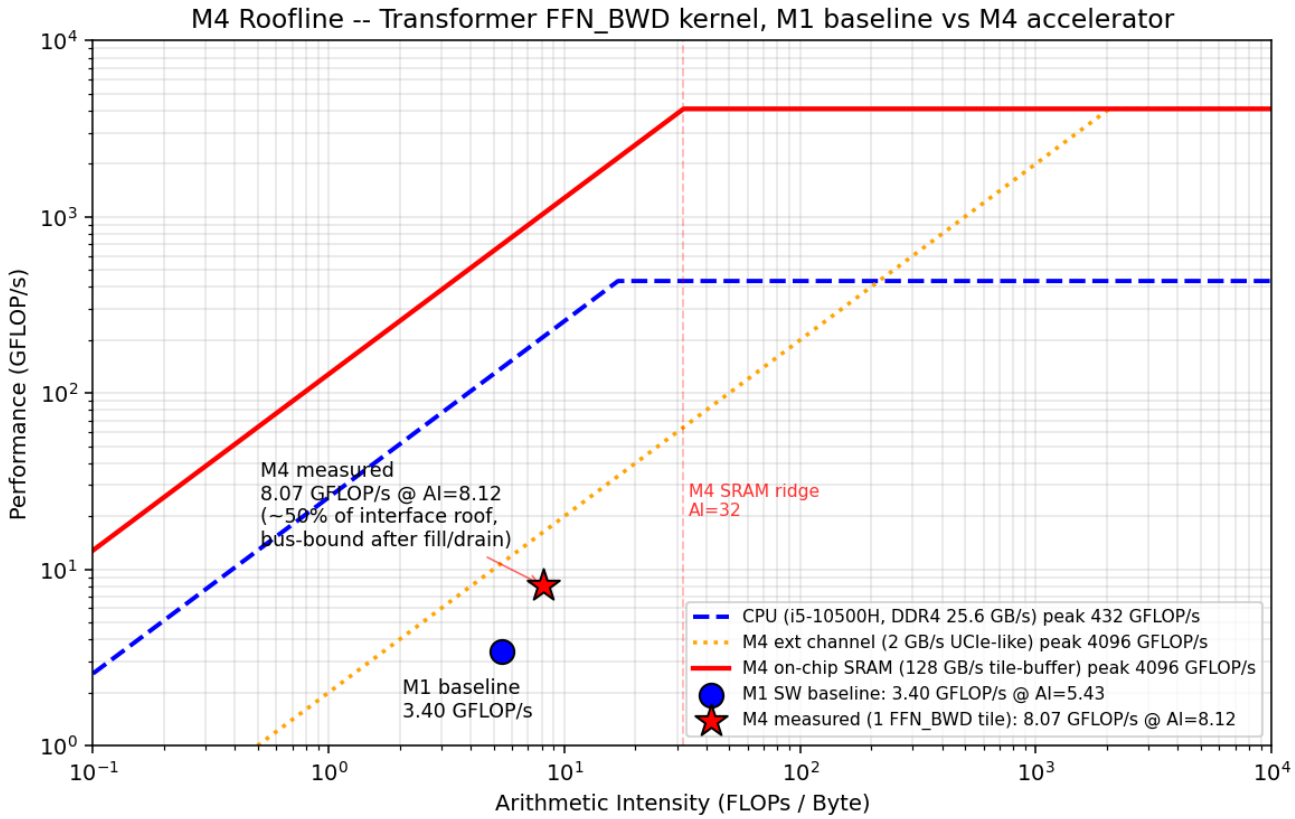

Figure 2 -- End-to-end UCIe waveform (§6).

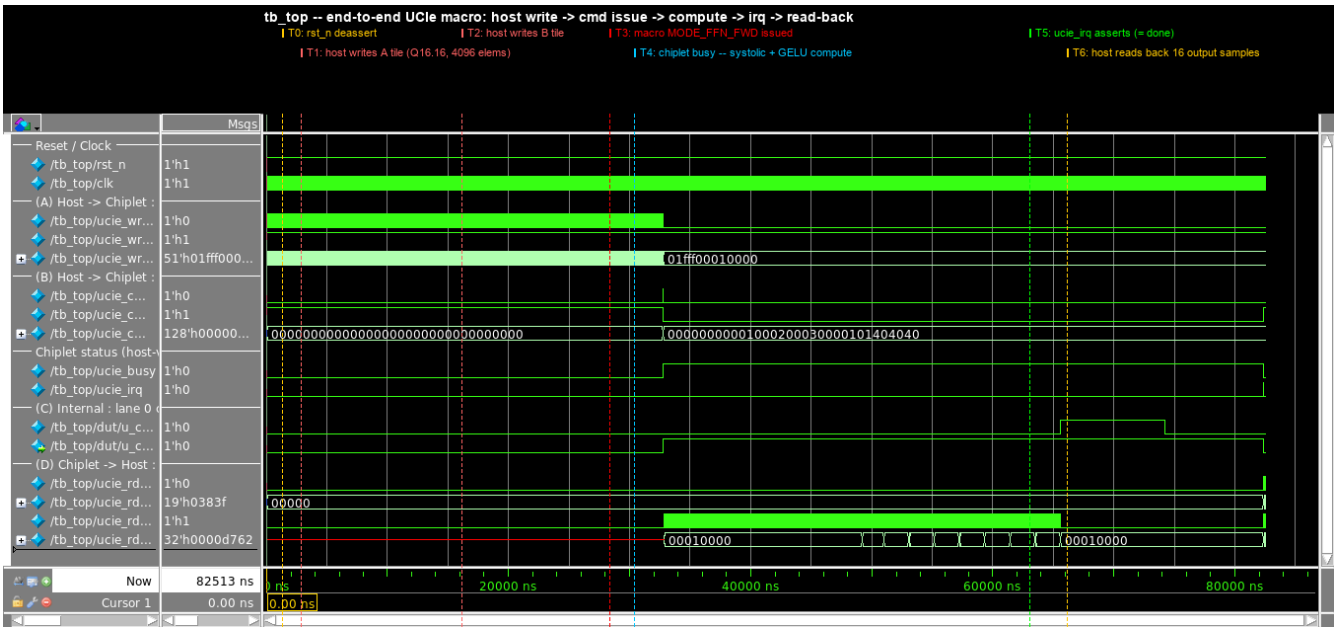

Figure 3 -- Chip block diagram (§4).

Figure 4 -- Systolic dataflow (§4).